Proof-of-Inference: Soundness, the Determinism Liveness Precondition, the Floating-Point No-Go, and the Routing/Chaining Extension Obligations

Companion soundness proofs for the Proof-of-AI execution verifier

Zoo Labs Foundation
research@zoo.ngo

Abstract

This note is the formal soundness companion to the Proof-of-AI execution verifier (the Proof-of-Inference primitive). It records, with theorems and proofs, four results. First, the *Freivalds soundness* of the matmul verifier over an exact integer accumulator: a fabricated output survives a beacon-derived challenge with probability at most $1/p$ per vector, hence $\leq 2^{-122}$ for two vectors over $\mathbb{F}_p$, $p = 2^{61} - 1$, with *zero* false rejection on honest re-derivation. Second, the *determinism contract as the liveness precondition*: the soundness theorem's "re-derive and compare" is well-defined only over a runtime that produces byte-identical traces, and we state exactly which clauses that requires and prove that under them the honest acceptance probability is 1. Third, a *floating-point no-go*: naive floating-point Freivalds is simultaneously unsound (a tolerance band is a profitable cheating corridor) and non-live (cross-hardware non-associativity false-rejects honest provers), and there is no single tolerance that escapes both; we then prove the *RepOps repair*—pinning the reduction order makes a floating-point GEMM a deterministic integer-mantissa computation, restoring exact Freivalds at a throughput cost. Fourth, the *extension obligations* that carry the primitive to the full model zoo: the routing proof (mixture-of-experts) and the sequential-chaining proof (linear-recurrent/state-space models and diffusion), stated as discharge conditions with correctness arguments. We are explicit about which results are backed by tested code (the Freivalds core and the two determinism prerequisites) versus designed-only (the commitment, the integration, and the two extensions).

Contents

1 Model and notation **2**

2 Freivalds soundness over the exact integer accumulator **2**

3 The determinism contract is the liveness precondition **3**

4 The floating-point no-go and the RepOps repair **4**
4.1 Why a floating-point GEMM is not a function of its operands 4
4.2 The RepOps repair: pinning the reduction order 5

5 Extension obligations: routing and chaining **6**
5.1 Mixture-of-experts: the routing proof 6
5.2 Linear-recurrent / SSM and diffusion: the sequential-chaining proof 6

6 What is proven on code versus on paper **7**

7 Conclusion **8**

1 Model and notation

We work over the prime field $\mathbb{F}_p$ with the Mersenne prime $p = 2^{61} - 1$. A layer's matrix product is $C = AB$ with $A \in \mathbb{Z}^{m\times k}$, $B \in \mathbb{Z}^{k\times n}$, $C \in \mathbb{Z}^{m\times n}$; the integer entries arise from the engine's quantized path, where i8×i8 products accumulate into i32 *without intermediate rounding.* A *challenge* fixes a random vector $r \in \mathbb{F}_p^n$, derived as $r = \mathrm{PRG}(\mathsf{ch} \,\|\, \ell)$ for layer index ℓ, where ch commits an unpredictable on-chain beacon (openBlockHash) so that, modeling the beacon as a random oracle, r is independent of any value the prover committed before the beacon was known. We treat r as uniform on $\mathbb{F}_p^n$ and independent of the prover's claimed output $\hat{C}$; this is the freshness property of the binding, taken as a hypothesis here (it is proved in the core paper's anti-replay lemma) and used as the soundness premise.

The verifier evaluates the Freivalds identity

$$A\,(B\,r) \;\stackrel{?}{=}\; C\,r \pmod p, \tag{1}$$

at cost $O(mk + kn + mn)$ field operations, one order below the $O(mnk)$ recomputation.

2 Freivalds soundness over the exact integer accumulator

Theorem 1 (One-layer soundness)**.** *Let A, B be the true operands and $\hat{C}$ the prover's claimed output, with $r \in \mathbb{F}_p^n$ uniform and independent of $\hat{C}$. Suppose p exceeds the magnitude of every entry of the honest product AB (so no honest entry wraps mod p). If $\hat{C} \neq AB$ as integer matrices, then*

$$\Pr_r\big[\,\hat{C}\,r \equiv A(Br) \pmod p\,\big] \;\le\; \frac{1}{p} \;=\; 2^{-61}.$$

Proof. Reduce entries mod p and set $D := \hat{C} - AB \in \mathbb{F}_p^{m\times n}$. Because p bounds the honest product's magnitude, an integer inequality $\hat{C} \neq AB$ cannot collapse to equality modulo p by wraparound, so $D \neq 0$ over $\mathbb{F}_p$. The check (1) accepts iff $\hat{C}r = (AB)r$, i.e. iff $Dr = 0$. As $D \neq 0$ it has a nonzero row D_i, and $Dr = 0$ forces $\langle D_i, r\rangle = 0$. Fix a coordinate j with $D_{ij} \neq 0$. Conditioning on $\{r_{j'}\}_{j'\neq j}$, the linear equation $\langle D_i, r\rangle = 0$ determines r_j uniquely, $r_j = -D_{ij}^{-1}\sum_{j'\neq j} D_{ij'}r_{j'}$ (the inverse exists, $\mathbb{F}_p$ a field, $D_{ij} \neq 0$). Since r_j is uniform over the p field elements and independent of the rest, it equals that one value with probability $1/p$. Hence $\Pr[Dr = 0] \le \Pr[\langle D_i, r\rangle = 0] = 1/p$. □ □

Corollary 2 (Zero false rejection)**.** *If $\hat{C} = AB$ as integer matrices, then* (1) *holds for* every *r. In particular an honest prover, and any honest re-deriving verifier on any hardware that computes the same integer product, passes with probability exactly* 1*.*

Proof. Integer equality $\hat{C} = AB$ implies entrywise equality mod p, so $\hat{C}r = (AB)r = A(Br)$ in $\mathbb{F}_p$ for all r. The premise that the integer product is hardware-independent is exactly the exactness of i8×i8→i32 accumulation (no rounding), so two honest machines agree on C bit-for-bit. □ □

Theorem 3 (Amplified soundness across a pass)**.** *Consider a forward or backward pass committing L matmul layers, verified with k_{rep} independent challenge vectors per checked layer and s layers sampled uniformly at random. Let a prover commit a trace that is not the genuine evaluation under the determinism contract.*

1. (Dense cheat.) *If the committed matmul output is wrong on every checked layer, the prover passes all checks with probability at most* $p^{-k_{\text{rep}}} \le 2^{-61k_{\text{rep}}}$*; for* $k_{\text{rep}} = 2$*, at most* 2^{-122}*.*

2. (Sparse cheat.) *If the prover cheats on a* ρ*-fraction of layers, the cheat escapes sampling with probability at most* $(1-\rho)^s$*; conditioned on a cheated layer being sampled it is caught except with probability* $p^{-k_{\text{rep}}}$*.*

3. (Output binding.) *A committed output hash inconsistent with the Freivalds-verified final-layer logits, under the pinned argmax/tie-break, is detected with certainty.*

Proof. For (1), the k_{rep} vectors on a discrepant checked layer are independent, so by Theorem 1 that layer passes with probability $\le (1/p)^{k_{\text{rep}}}$; a dense cheat is discrepant on every checked layer, so the prover must survive at least one such check, bounded by $p^{-k_{\text{rep}}}$. For (2), the s sampled layers miss the cheated fraction with probability at most $(1-\rho)^s$ (sampling without replacement only lowers this); if any cheated layer is sampled, (1) applies to it. For (3), the final pre-softmax logits are a matmul output, hence Freivalds-checked; the verifier recomputes the argmax under the pinned tie-break (Section 3) and compares to the committed token—both are deterministic functions of the verified logits, so any inconsistency is detected with probability 1. □ □

Remark 1 (Spot-check to certainty)**.** *The residual sparse-cheat term* $(1-\rho)^s$ *does not reach* 0 *by sampling alone. An interactive bisection over the committed trace—*$O(\log L)$ *rounds isolating a single disputed matmul whose inputs both parties agree on, then resolved by one on-chain re-execution or Freivalds check—upgrades part (2) to detection with* certainty *at logarithmic on-chain cost. This is orthogonal to the field soundness above and is recorded in the core paper.*

3 The determinism contract is the liveness precondition

Theorem 1 and Corollary 2 presuppose that two honest evaluations agree. That is a property of the runtime, not of the algebra, and it is precisely the precondition that makes the verifier *live* (honest provers are accepted) as opposed to merely *sound* (cheaters are rejected). We state it as a contract and prove honest acceptance under it.

Definition 1 (Determinism contract)**.** *A runtime satisfies the determinism contract for a model if, digested into* runtimeMeasurement *and gated by governance, it fixes: (D1)* ***integer matmul*** *on the i8 (or lower) path with exact i32 accumulation; (D2)* ***temperature-0 argmax*** *decoding (no stochastic sampling on the proof path); (D3) a* ***specified tie-break****—the* lowest token id *wins an exact-logit tie; (D4)* ***pinned kernels and tokenizer****, identified in* runtimeMeasurement*; (D5) a* ***committed seed*** *for any residual stochastic op (inert on the greedy path).*

Theorem 4 (Honest acceptance / liveness)**.** *Let two honest provers evaluate the same model on the same input under a runtime satisfying Definition 1. Then they produce byte-identical activation traces and the identical output, and each passes every Freivalds challenge with probability* 1*.*

Proof. By (D1) every matmul output is an exact integer matrix, identical across hardware (i8×i8→i32 has no rounding and integer arithmetic is associative and machine-independent), so all committed matmul leaves agree. The non-matmul ops (normalization, activation, softmax) are deterministic elementwise functions of their committed (and agreeing) inputs under the pinned kernels (D4), so their committed leaves agree. Decoding is arg max over the agreeing final logits (D2); ties—the only point of potential ambiguity—are resolved identically by the fixed lowest-id rule (D3); (D5) consults no RNG on the greedy path. By induction over the layer sequence the two traces are

byte-identical and the output tokens coincide. Acceptance with probability 1 is then Corollary 2 applied at each checked layer. □ □

Remark 2 (Tested in code)**.** *Clauses (D1), (D3), and the routing analog of (D3) are implemented and unit-tested in the inference engine: the exact-integer Freivalds core (`poi.rs`, 6/6 tests), the lowest-token-id argmax tie-break (`sampler.rs`, replacing Rust's last-maximal `max_by`), and the canonical backend-independent MoE top-k (`ops.rs`, `set_moe_proof_deterministic` / `deterministic_topk_indic`. The trace commitment and the forward-pass integration that stream the leaves are designed, not yet built; Theorem 4 is therefore a statement about the contract a built runtime must meet, two of whose three nontrivial clauses are already enforced.*

4 The floating-point no-go and the RepOps repair

We now prove the central negative result motivating the quantized proof-bearing mode: *naive floating-point Freivalds cannot be both sound and live.* Then we prove the positive repair.

4.1 Why a floating-point GEMM is not a function of its operands

Definition 2 (Reduction order)**.** *A floating-point dot product of length k is computed by summing the k rounded products in some order specified by a binary* reduction tree τ *on the indices, with an IEEE round-to-nearest-even after each node. Two kernels may use different τ (tiling, thread count, FMA fusion, library version), and floating-point addition is non-associative: $(a+b)+c \neq a+(b+c)$ in general.*

Proposition 5 (Cross-hardware divergence)**.** *There exist bf16 operands A, B and reduction trees $\tau \neq \tau'$ such that $\mathrm{GEMM}^{\tau}(A,B) \neq \mathrm{GEMM}^{\tau'}(A,B)$. Consequently two honest provers using kernels with different reduction orders commit different outputs $C \neq C'$.*

Proof. Non-associativity of IEEE addition gives a length-3 witness: choose a, b, c representable in the format with $\mathrm{fl}(\mathrm{fl}(a+b)+c) \neq \mathrm{fl}(a+\mathrm{fl}(b+c))$ (the standard cancellation example, where a is large, $b=-a$, and c is below the rounding unit of a). Embedding this triple as one row of A and one column of B realizes a GEMM entry whose value differs between the left-leaning and right-leaning trees. The two trees are both legitimate honest kernels, so $C \neq C'$. □ □

Theorem 6 (Floating-point no-go)**.** *Consider verifying a floating-point GEMM by the rule "accept iff $\|Cr - A(Br)\|_\infty \leq \epsilon$" for a fixed tolerance $\epsilon \geq 0$, where C is the prover's claimed output and the verifier re-derives $A(Br)$ with its own (honest) reduction order. Then:*

1. (Liveness requires ϵ large.) *If $\epsilon < \delta_{\mathrm{hw}}$, where $\delta_{\mathrm{hw}} = \max\|C - C'\|$ over honest cross-hardware reduction orders, then some honest prover is rejected. By Proposition 5 $\delta_{\mathrm{hw}} > 0$, so $\epsilon = 0$ is not live.*

2. (Soundness requires ϵ small.) *For any $\epsilon > 0$ there is a cheating $\hat{C} \neq AB$ accepted with probability 1 over r: namely any $\hat{C}$ with $\|(\hat{C} - AB)r\|_\infty \leq \epsilon$ for the verifier's r. The set of such $\hat{C}$ has nonzero measure and contains points arbitrarily far from AB in entries orthogonal to r.*

3. (No safe ϵ.) *If the profitable-cheat margin δ_{cheat} (the smallest $\|(\hat{C} - AB)r\|$ a value-skipping prover can achieve) satisfies $\delta_{\mathrm{cheat}} \leq \delta_{\mathrm{hw}}$, then* no ϵ *is both live and sound.*

Proof. (1) If $\epsilon < \delta_{\text{hw}}$, pick honest orders realizing $\|C - C'\| = \delta_{\text{hw}}$ (Proposition 5); the prover commits C, the verifier re-derives with the order giving C', and $\|Cr - A(Br)'\|$ can exceed ϵ, rejecting honest work. (2) Fix the verifier's r. The constraint $\|(\hat{C} - AB)r\|_\infty \le \epsilon$ is a band around the hyperplane $\{D : Dr = 0\}$ in the mn-dimensional space of discrepancy matrices $D = \hat{C} - AB$; it is satisfied by a positive-measure set, and because r projects D to $Dr \in \mathbb{F}_p^m$ ($m \ll mn$ constraints), the band contains D with arbitrarily large entries in the $(mn - m)$ directions orthogonal to the row span of the projection. Any such $\hat{C} = AB + D$ is a genuine cheat accepted for that r. (3) Live demands $\epsilon \ge \delta_{\text{hw}}$ (part 1); sound demands $\epsilon < \delta_{\text{cheat}}$ (else a profitable cheat lands in the band, part 2). If $\delta_{\text{cheat}} \le \delta_{\text{hw}}$ the two intervals are disjoint and no ϵ satisfies both. □ □

Remark 3 (Interpretation)**.** *The no-go is the formal content of "do not run Freivalds on floating point." The corridor of part (2) is the cheating manifold a tolerance opens; part (1) is the false-reject the same tolerance is forced to admit to keep honest cross-hardware provers alive; part (3) says the adversary's job (make Dr small) is no harder than the honest hardware's unavoidable drift, so the two cannot be separated by a threshold. The integer accumulator escapes the trilemma by making $\delta_{\text{hw}} = 0$ exactly (Corollary 2), which collapses the band to the point $\{Dr = 0$ with $\epsilon = 0\}$ and restores Theorem 1.*

4.2 The RepOps repair: pinning the reduction order

Proposition 7 (Floating-point GEMM is exact integer-mantissa arithmetic at fixed order)**.** *Fix an IEEE format F with round-to-nearest-even and fix a reduction tree τ. Then $\text{GEMM}_F^\tau(A, B)$ is a deterministic function of (A, B), computable by integer arithmetic on the significand/exponent fields with a deterministic rounding step at each tree node, and identical on every machine that honors F and τ.*

Proof. Each product $a_i b_j$ has significand equal to the integer product of the two integer significands and exponent equal to the integer sum of exponents—exact in $\mathbb{Z}$. A node combines two partial sums $u = s_u 2^{e_u}$, $v = s_v 2^{e_v}$ (integer s): align exponents by an integer shift, add significands by an integer add, apply round-to-nearest-even (a total function of the exact integer sum and F's precision). Each operation is integer arithmetic plus a deterministic rounding decision, so the node output is a deterministic function of (u, v). Induction over τ gives the claim; the disagreement of Proposition 5 is exactly a difference in τ, removed by fixing it. □ □

Theorem 8 (RepOps soundness)**.** *Pin τ in* runtimeMeasurement *and run the Freivalds check over the integer significand–exponent representation of $\text{GEMM}_F^\tau(A, B)$, with p chosen to bound the accumulated significand range. Then the check has $\delta_{\text{hw}} = 0$ (zero false rejection across hardware honoring F, τ) and per-vector soundness $\le 1/p$, i.e. Theorem 1 and Corollary 2 hold verbatim for the floating-point tier.*

Proof. By Proposition 7 every honest machine computes the identical rounded output, so $\delta_{\text{hw}} = 0$ and Corollary 2 applies. The pinned-order output is an exact integer (significand $\times 2^e$) per entry; treating these integers as field elements mod p (with p bounding their magnitude), a fabricated $\hat{C} \ne \text{GEMM}_F^\tau(A, B)$ gives $D \ne 0$ over $\mathbb{F}_p$, and Theorem 1's argument is unchanged. □ □

Remark 4 (Cost, and why RepOps is a tier not a default)**.** *Pinning τ forfeits the vendor kernel's freedom to choose the reduction order, which is the source of its speed; RepOps therefore runs slower than the vendor optimum (the standard reproducible-summation trade-off). Hence the floating-point tier offers RepOps-Freivalds* or *a TEE/CC attestation (hardware attests the unpinned fp run, no τ pin, at the cost of a hardware-trust assumption), and the quantized tier—the default, and what the* Zoo *zoo ships—needs neither pin nor hardware because integer accumulation is already exact.*

5 Extension obligations: routing and chaining

The base primitive verifies matmuls. Two model-structural features depart from a plain GEMM stack, and each is discharged by one additional technique. We state each as a proof obligation with a correctness argument; both are designed, not yet built (the routing *determinism* prerequisite is built and tested).

5.1 Mixture-of-experts: the routing proof

Proof obligation 1 (Routing proof)**.** *For each routed token, the prover commits (i) the router logits g (a matmul output), (ii) the selected expert index set $S = \mathrm{top}_k(g)$ under the canonical tie-break (score descending, expert-index ascending), and (iii) the sparse dispatch pattern (which tokens enter which experts). A verifier challenging the token: Freivalds-checks g; recomputes $\mathrm{top}_k(g)$ from the committed g under the canonical tie-break and checks it equals S; and checks that the committed expert-FFN matmuls used exactly the experts in S on exactly the dispatched tokens.*

Proposition 9 (Routing soundness, given Obligation 1)**.** *Under the determinism contract extended with the canonical MoE top-k (built; Section 3), a prover who routes to a cheaper or wrong expert is detected except with the base Freivalds error.*

Proof. A different selected expert is a different sub-network. Two disjoint cheats are possible. (a) *Wrong indices*: the prover commits $S' \neq \mathrm{top}_k(g)$. Since g is Freivalds-verified and the canonical tie-break is a deterministic total function of g (built), the verifier's recomputation yields $\mathrm{top}_k(g) \neq S'$ and the check fails with certainty. (b) *Wrong matmul*: the prover commits the correct S but evaluates a different (cheaper) expert FFN. That FFN's output is a matmul whose committed operands are the designated expert's weights; the Freivalds check on it catches the substitution except with probability $\leq p^{-k_{\mathrm{rep}}}$ (Theorem 1, since a different sub-network gives a different product). Determinism of the top-k removes the false-reject that a non-canonical tie-break would otherwise cause for an honest prover. □ □

5.2 Linear-recurrent / SSM and diffusion: the sequential-chaining proof

Proof obligation 2 (Sequential chaining)**.** *For a layer carrying a state $S_t = \Phi_t S_{t-1} + \Psi_t$, $o_t = \Omega_t S_t$ (the recurrent/SSM case) or a denoising loop with latent $z_t = f_\theta(z_{t-1}, t)$ (the diffusion case), the prover commits the ordered states $S_0, \ldots, S_T$ (resp. latents $z_0, \ldots, z_T$) into the trace, and the per-step check at step t takes its input state from the committed step-$(t-1)$ output leaf. A verifier challenging step t checks both the step's matmuls (Freivalds) and that the input state it used is byte-equal to the committed step-$(t-1)$ output leaf.*

Proposition 10 (Chaining soundness, given Obligation 2)**.** *A prover who computes each step honestly in isolation but splices a fabricated intermediate state (resp. latent) at a step boundary is detected with certainty.*

Proof. Per-step Freivalds catches an arithmetically wrong step. The remaining cheat is a *seam* break: step t is internally consistent with respect to some $\tilde{S}_{t-1}$ that step $t-1$ did not actually produce. Obligation 2 pins step t's input to the committed step-$(t-1)$ output leaf, so $\tilde{S}_{t-1} \neq S_{t-1}^{\mathrm{committed}}$ is a mismatch between the leaf step t consumed and the leaf step $t-1$ produced; the verifier compares the two committed leaves and detects the inequality with certainty (it is an equality of committed bytes, not a probabilistic check). For diffusion, the additional determinism clauses—pinned noise schedule and sampler/seed in runtimeMeasurement—make the trajectory a deterministic function

of the prompt and the committed seed, so honest re-derivation of any challenged step reproduces the committed latent and does not false-reject. The recurrent and diffusion cases are identical in form (a per-step linear/parametric map plus a committed-leaf seam), so one technique discharges both. □ □

Theorem 11 (Whole-zoo coverage, modulo the obligations)**.** *With the base primitive (Theorems 1, 3) under the determinism contract (Theorem 4), the routing proof (Proposition 9), and the sequential-chaining proof (Proposition 10), every architecture family in the* Zoo *zoo—dense, mixture-of-experts, linear-recurrent/state-space, multimodal vision (a ViT tower whose convolution is im2col→GEMM), audio (conformer GEMMs with a committed, recomputed FFT/mel front end), and diffusion/Transfusion—has a complete soundness account. No third core verification technique is required.*

Proof. Each family's heavy arithmetic is dense matmul (attention projections and MLPs; expert FFNs; the per-step maps Φ_t, Ψ_t, Ω_t or the per-step denoiser; the ViT tower including the im2col-expressed convolution; the conformer blocks), discharged by Theorems 1–3 under Theorem 4. Sparse expert routing, when present, is discharged by Proposition 9; an inter-step state or latent seam, when present, by Proposition 10. The residual per-modality operations (tokenization, image preprocessing, FFT/mel feature extraction, noise scheduling) are deterministic, pinned, input-independent or elementwise, hence committed and recomputed with zero false rejection under the determinism contract, and the per-modality input/output hashes bind the boundary data. Every family reduces to these cases plus pinned pre/post-processing, so the stated techniques suffice. □ □

6 What is proven on code versus on paper

Result	Status	Evidence
Freivalds one-layer + amplified soundness (Thm. 1, 3)	built core, proven	`poi.rs` 6/6 tests
Zero false rejection on exact integers (Cor. 2)	built core, proven	`test_honest_matmul_passes`
Sparse-cheat detection (Thm. 3.2)	built core, proven	`test_larger_random_matmul_catches_sparse`
Signed-entry field reduction	built core, tested	`test_negative_entries_field_reduction`
Deterministic challenge derivation (freshness half)	built core, tested	`test_challenge_derivation_deterministic`
Fail-closed shape contract	built core, tested	`test_wrong_dims_rejected`
Lowest-id argmax tie-break (D3)	built, tested	`sampler.rs argmax_f32`
Canonical MoE top-k (routing determinism)	built, tested	`ops.rs deterministic_topk_indices`
Determinism contract ⇒ liveness (Thm. 4)	contract; 2/3 nontrivial clauses built	D1, D3 + MoE-D3 in engine

Result	Status	Evidence
Floating-point no-go (Thm. 6)	proven (negative result)	this document
RepOps repair (Prop. 7, Thm. 8)	designed, proven	this document
Routing proof soundness (Prop. 9)	designed, proven	Obligation 1
Chaining proof soundness (Prop. 10)	designed, proven	Obligation 2
Activation-trace MMR commitment + forward-pass integration	designed, not built	core paper

Table 1: Soundness results by status. "Built" = implemented and tested in the inference engine; "designed, proven" = formally argued here, implementation pending; "proven (negative)" = an impossibility we rely on to justify the quantized design.

7 Conclusion

The Proof-of-Inference primitive is sound where it is built and proven where it is designed. The matmul verifier catches a fabricated output with probability $\geq 1 - 1/p$ per challenge ($\geq 1 - 2^{-122}$ at two vectors) and never false-rejects honest work, *because* it runs over an exact integer accumulator; this is implemented and 6/6 unit-tested. The determinism contract is the liveness precondition that makes "re-derive and compare" well-defined, and its two nontrivial decision points—argmax ties and MoE routing ties—are pinned in code with canonical tie-breaks. The reason the proof-bearing mode is quantized is not preference but a genuine impossibility: floating-point Freivalds cannot be simultaneously sound and live, and the only repair (pinning the reduction order) trades exactness back in at a throughput cost. Finally, the full model zoo is covered by exactly two designed extensions—a routing proof for mixture-of-experts and a single sequential-chaining proof shared by the recurrent/state-space family and the diffusion loop—each proven sound modulo its discharge obligation. The honest remaining work is the commitment layer, the forward-pass integration, and those two extensions; the mathematical core they will be wired into is settled.